

ADAPTIVE LIGHTING AND WINDOW SYSTEM PROJECT REPORT

25MTRN04H-INDUSTRIAL INTERNET OF THINGS

SUBMITTED BY:

ANAS HATEM MAHDALI - 239045

MODULE STAFF:

PROF. MOSTAFA ABDELAZIZ

ENG. ABDELRAHMAN MOHAMMED

ENG. YOUSSEF MAXI

Table of Contents

Introduction..... 1

Perception Layer 1

 Sensors 1

 Actuators 2

Sensor Calibration & Signal Processing 3

Control Systems & State Management 3

 Tier 1: Absolute Failsafe Override (Emergency Protocol) 3

 Tier 2: Automated Control (Auto Mode)..... 3

 Tier 3: Manual Override 4

Network Architecture & Telemetry Pipeline 4

The 3D Digital Twin (UI) 5

Critical Power Management Considerations..... 5

Robustness 6

Conclusion 6

References..... 6

Introduction

Every day, almost every household has at least one occupant that will at some point forget the light on even if for a few minutes, another might even forget their fan on which is an even greater waste of energy. The article “How much light energy waste are UK households responsible for?” (100green(2023)) states that the average wasted kWh per household is 12.05kWh per year and that is just for the household lightbulb, this amount of energy can power a typical home for 12 house under essential loads.

This project describes the design and development of an IoT based centralized mechatronics system for autonomous lighting and window control. The system uses an ESP32 microcontroller to collect real-time data from the environment and to control physical components in order to ensure the best conditions in the room. A major element of this design is the ability to send and receive data in real time – giving operators access to up-to-the-minute information – as well as real-time remote manual override functions, with an inbuilt failsafe system for gas emergencies.

The system is mostly based on an MQTT protocol interface with the Ubidots cloud platform, which is used for telemetry and remote control. Moreover, the system is also configured with an additional HTTPS pipeline that sends the data to get logged to google bigquery and an additional feature is a local websockets architecture with more visualization capabilities than the ubidots dashboard

Perception Layer

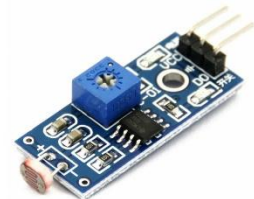
Sensors

The system uses the ESP32 as the main processor and the following sensors are used to monitor the environment



- **DHT22 (Temperature & Humidity - Pin 32):** Selected for its reliable digital output and accuracy compared to the LM35, it is also sold pre-calibrated which saves a lot of headaches

- **LDR 1 & 2 (Photoresistors - Pins 34, 35):** Analog light sensors which detect ambient light intensity. LDR1 monitors exterior light to identify the day/night cycle and automate the curtain mechanism. LDR2 monitors interior ambient light of the room.





- **PIR Sensor (Passive Infrared - Pin 25):** Passive Infrared motion sensors detect movement by measuring changes in infrared radiation of nearby objects, it is used for localized automated lighting control.

- **MQ5 Gas Sensor (Combustible Gas - Pin 33):** Analog sensor used to detect even trace amounts of LPG, natural gas, methane and propane. Critical for the system's emergency failsafe protocol.



Actuators

- **Window Motor Mechanism (Pins 26, 27):** Driven by an L298N Motor Driver. Operates on a timed continuous rotation found through calibration with distances and available spaces



- **Curtain Motor Mechanism (Pins 18, 19):** Driven by an L298N Motor Driver. Operates on a 300ms hardware timer precisely calibrated for the available space and number of revolutions required

- **L9110 Fan Module (Pins 22, 23):** Directional DC fan for active ventilation in order to expel harmful gases and to lower room temperature to a comfortable degree





- **3W LED Module (Pin 2):** Used as the primary lighting source for occupants of the room, emits from 180 to 200 lumens which is enough for the size of the room used

Sensor Calibration & Signal Processing

Raw data from the sensors needs to be calibrated and conditioned so that it is available to the control logic and is easily digestible.

- **MQ5 Gas Calibration:** MQ5 needs to be powered for at least 2 minutes to stabilize the internal heating coil. In software, the analog reading is converted to a percentage ($\text{value} / 4095.0$) * 100 to get an analog reading from the 12-bit analog input
- **Luminosity Mapping:** The 12-bit LDR reading is inverted and mapped to an estimated Lux value to be used for human-readable display on the dashboard. The analog reading for daytime is set to ≤ 2000 .
- **Signal Noise Filtering:** There is electrical noise from physical motors and analog sensors vary naturally. A hysteresis was added to the software logic to eliminate the fast switching, so that the emergency state is only activated at 60% PPM, and the emergency state will only be released on a drop to 40% PPM

Control Systems & State Management

The firmware for the ESP32 has a multi-level structure in order to handle the system states and avoid logical conflicts between human inputs and sensor data.

Tier 1: Absolute Failsafe Override (Emergency Protocol)

There's more to logic than just being safe. If the MQ5 detects gas concentrations exceeding **60%**, the system enters an absolute lockout.

- Ignores all user inputs and automated temperature/light rules.
- It activates the cooling fan and automatic window to open for better air circulation
- To avoid motor stuttering due to sensor jitter, the system goes into emergency-mode and will not return control to the user until the gas concentration is safely below 40%

Tier 2: Automated Control (Auto Mode)

The system works on its own under safe conditions:

- **Thermal Regulation:** If temperature is above 30.0°C, the window opens and the fan activates to ventilate the room

- **Light Regulation:** If outside light crosses the 2000 threshold that indicates daytime, the curtain automatically slides to open the window and power off the bulb to save energy

Tier 3: Manual Override

Users can disable auto mode via the dashboard.

- The ESP subscribes to MQTT topics to receive target states for window, curtain and fan operation, overriding local operation
- The motor timers (windowMoving, curtainMoving) control when the logical state variable flips (e.g.: windowState = 1) to keep the hardware in sync
- The dashboard updates the 3D graphics as soon as a button is clicked and the ESP32 handles the physical actuation in the background.

Network Architecture & Telemetry Pipeline

A ubidots dashboard was developed on this system, but to store the data in the cloud, the transmitted data was first passed through the google cloud platform and handled using HTTPS, and also to enable real-time control and bi-directional communication between the dashboard and the ESP32 a websocket was used, a simple dashboard was provided by ubidots as well.

The system utilizes a branched data pipeline to handle both real-time control and long-term data storage.

- **Pipeline 1: Ubidots MQTT:** the esp publishes telemetry every 5 seconds to the esp32-system device label, it also keeps listening to last value topics to receive state changes from the dashboard, the topics subscribed to from the esp are:
 - /v2.0/devices/esp32-system/system-mode/lv
 - /v2.0/devices/esp32-system/window-control/lv
 - /v2.0/devices/esp32-system/curtain-control/lv
 - /v2.0/devices/esp32-system/fan-control/lv

And the topics which the ESP publishes to are:

- /v2.0/devices/esp32-system/temperature
- /v2.0/devices/esp32-system/ldr-outside
- /v2.0/devices/esp32-system/gas-level
- /v2.0/devices/esp32-system/window-status
- /v2.0/devices/esp32-system/curtain-status
- /v2.0/devices/esp32-system/fan-status
- **Pipeline 2: Long-Term Cloud Storage:** The ESP32 makes a secure HTTPS get request to a Google Apps Script Web App every 60 seconds. Parses URL parameters (temp, lux, gas, led status) and commits a dataset containing the data and the timestamp into a Google Cloud BigQuery dataset to be used for future data analysis..

- **Pipeline 3: Low-Latency WebSockets:** To be used as alternative dashboard, the ESP32 is connected to the local Node.js WebSocket Broker that is used on the pipeline 3 named Low-Latency WebSockets. This broker has a secure ngrok tunnel exposed to the internet. This was achieved by using a 3D Digital Twin that was driven by a serialized JSON payload sent from the ESP32 every 2 seconds, avoiding blocks imposed by browser CORS restrictions (Cross-Origin Resource Sharing), a security feature of the browser which prevents scripts from one origin from making requests to another via a local http server

The 3D Digital Twin (UI)

The user interface abandons traditional 2D dashboards and introduces a WebGL and Three.js based First-Person 3D Virtual Control Room

- **Real-Time Mirroring:** The 3D scene is rendered to produce the state of physical hardware dynamically. Virtual fan rotates constantly when physical fan operates, virtual window and virtual curtains move along Z and X axes to synchronize with the physical motor timers
- **Dynamic Environments:** The Three.js scene dynamically changes to high resolution texture maps (Day/Night landscapes) outside the virtual window using the sensor data collected by the LDR sensor, which simulates real lighting.
- **CORS Bypass:** To load local high-resolution textures without triggering browser Cross-Origin Resource Sharing (CORS) security blocks, the HTML file is served via a local Node http-server. To avoid browser Cross-Origin Resource Sharing (CORS) security blocks for loading high-resolution local textures, the HTML file is being served from a local Node http-server

Critical Power Management Considerations

The MQ5 sensor has considerable power consumption as a result of its internal heating element (~150mA continuous). The ESP32 turns on the MQ5 heater, two DC gear motors (window and curtains) and the ventilation fan simultaneously during a gas emergency.

The logic level (ESP32) needs to be separated from the load level (L298N motor drivers) because if the voltage drops to a catastrophic level, the ESP32 will reset, and the motor drivers will enter an infinite loop. There is an independent 9V/12V high-current power supply with common ground connected to the microcontroller, providing power to the motor drivers

Robustness

It is resilient in the sense that in the event of connection failure, the system continues to offer the same automatic functionality, avoiding system failure.

The system actively watches the Wi-Fi link and the MQTT connection: if either one is down, the esp will reset the automode to the true state, so the system will not get in a loop waiting for cloud commands that will no longer be received

```
void loop() {  
  if (WiFi.status() != WL_CONNECTED) {  
    autoMode = true;  
    static unsigned long lastWifiAttempt = 0;  
    if (millis() - lastWifiAttempt > 10000) {  
      WiFi.reconnect();  
      lastWifiAttempt = millis();  
    }  
  } else {  
    if (!ubidots.connected()) {  
      autoMode = true;  
    }  
  }  
}
```

Conclusion

The system is able to show the feasibility of low cost microcontroller integration with sophisticated cloud based telemetry and 3D visualization. The system's energy-saving capabilities are achieved through a tiered control architecture and the use of the ESP32, overcoming the problems of energy waste identified in the introduction and making the living area responsive and energy-efficient.

All primary design objectives are addressed: autonomy, safety and manual transparency. The local environmental regulation is maintained even if the network is unstable, with the so called "Offline Resilience" function of the system. This prototype is a scalable model for energy management in today's smart home, demonstrating that smart energy management can be realized without sacrificing user safety or the lack of operational oversight.

References

How much light energy waste are UK households responsible for?. 100Green. (2023, September 14). <https://100green.com/blog/news/uk-light-energy-waste/>